Design and Analysis of Algorithms

ASSIGNMENT 1 (CO1)

Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

Name	Avishek N
Register No	192511009
Institution	Saveetha Institute of Medical and Technical Sciences (SIMATS Engineering)
Course	Design and Analysis of Algorithms
Random seed	1009 (last 4 digits of register number)
Algorithms studied	Linear search (iterative) · Binary search (recursive) · Hash table with chaining (Group A third strategy)
GitHub repository	https://github.com//DAA-A1-192511009-Avishek
Language / platform	Python 3, standard library only for the algorithms

Contents: Task 1 Design · Task 2 Basic operation · Task 3 Time-complexity derivation · Task 4 Best/average/worst case · Task 5 Asymptotic analysis · Task 6 Recurrence formulation · Task 7 Recurrence solution · Task 8 Implementation and testing · Task 9 Experimental performance · Task 10 Theory vs experiment · Task 11 Final recommendation · Task 12 Day-4 design challenge (A–D)

Task 1 – Design the Algorithm

Problem. Given n student records, each with a unique registration number, decide whether a given key exists and return its position. I chose three strategies that rely on three different ideas: scanning everything, repeatedly halving a sorted range, and computing the location directly from the key.

	Algorithm 1: Linear search	Algorithm 2: Binary search	Algorithm 3: Hash table (chaining)
Design strategy	Brute force	Decrease-and-conquer (decrease by a factor of 2)	Space–time trade-off (representation change)
Style	Iterative	Recursive	Iterative (build and lookup)
Basic idea	Compare the key with each record in turn until it is found or the list ends.	Compare with the middle record of a sorted array; the answer can only lie in one half, so discard the other half and repeat.	Store each key in bucket (key mod m). A lookup only inspects the one short chain in that bucket.
Preprocessing	None; works on raw, unsorted data.	Sort the array once.	Build the table once (n insertions, table size m ≈ n/0.75).
Why chosen	Simplest possible baseline and the only option when data is unsorted and queries are rare.	Classic logarithmic method; the sort can be paid once and reused by every later search.	Aims for constant-time lookup; useful when the number of searches is very large compared with the number of updates.

Pseudocode

```
ALGORITHM LinearSearch(A[0..n-1], key)
    for i ← 0 to n-1 do
        if A[i] = key then return i
    return -1

ALGORITHM Preprocess(A[0..n-1])           // done once
    sort A in ascending order

ALGORITHM BinarySearch(A, key, lo, hi)   // first call: lo = 0, hi = n-1
    if lo > hi then return -1            // base case: empty range
    mid ← ⌊(lo + hi) / 2⌋
    if A[mid] = key then return mid
    else if key < A[mid] then return BinarySearch(A, key, lo, mid-1)
    else                      return BinarySearch(A, key, mid+1, hi)

ALGORITHM BuildTable(A[0..n-1])           // done once
    m ← odd integer close to n / 0.75
    create m empty chains
    for each key in A do append key to chain[key mod m]

ALGORITHM HashSearch(key)
    for each x in chain[key mod m] do
        if x = key then return FOUND
    return NOT FOUND
```

Task 2 – Identify the Basic Operation

In all three algorithms the dominant work is **comparing the search key with a stored registration number**. Every other statement (index arithmetic, loop control, computing key mod m) is done a constant number of times per key comparison, so counting comparisons gives the growth of the running time without depending on the machine.

Algorithm	Basic operation and where it occurs	Why it is suitable	How its count changes as n grows
Linear	`if A[i] = key` inside the for loop.	Executes once per loop iteration, so it dominates the loop and is the only thing that depends on the data size.	Can go up to n: a larger database means proportionally more records to scan.
Binary	`if A[mid] = key / key < A[mid]` in each recursive call (counted as one three-way comparison).	One comparison is made per call and each call discards half the range, so the number of calls is the number of comparisons.	Grows only by 1 each time n doubles; n = 1,000 needs about 10 and n = 1,000,000 about 20.
Hash	`if x = key` while walking the chain of bucket key mod m.	The hash computation is constant; the only variable cost is the length of the chain inspected.	Chain length depends on the load factor α = n/m, not on n, so it stays almost constant as n grows.

Evidence – the basic operation highlighted in my code (yellow = key comparison and its counter):

```
5     def linear_search(arr, key):
6         """Returns (index or -1, number of key comparisons)."""
7         comparisons = 0
8         for i in range(len(arr)):
9             comparisons += 1
10            if arr[i] == key:
11                return i, comparisons
```

Figure 2.1: Linear search – basic operation (algo1_linear.py)

```
8     def binary_search(arr, key, lo=0, hi=None, _c=None):
9         """Returns (index or -1, number of key comparisons)."""
10        if hi is None:
11            hi = len(arr) - 1
12        if _c is None:
13            _c = [0]
14        if lo > hi:                         # base case: empty range -> not found
15            return -1, _c[0]
16        mid = (lo + hi) // 2
17        _c[0] += 1                          # one comparison per call
18        if arr[mid] == key:
19            return mid, _c[0]
20        if key < arr[mid]:
21            return binary_search(arr, key, lo, mid - 1, _c)
22        return binary_search(arr, key, mid + 1, hi, _c)
23
24
```

Figure 2.2: Binary search – basic operation (algo2_binary.py)

```
19        def search(self, key):
20            """Returns (found?, number of key comparisons)."""
21            b = self.buckets[key % self.m]
22            if b is None:
23                return False, 0
24            c = 0
25            for k in b:
26                c += 1
27                if k == key:
28                    return True, c
29            return False, c
```

Figure 2.3: Hash table – basic operation (algo3_hash.py)

Task 3 – Derive the Time-Complexity Function

Input size: n = number of student records (registration numbers) stored. Key comparisons are counted by C(n).

Algorithm 1 – Linear search (no preprocessing)

The loop body runs once for every position inspected, and the comparison is executed once per iteration.

$$C_{worst}(n) = \Sigma_{i=0}^{n-1} 1 = n$$

If the key is present and equally likely to be in any of the n positions, a key at position i (1-based) costs i comparisons:

$$C_{avg,found}(n) = (1/n)\ \Sigma_{i=1}^{n} i = (1/n) \cdot n(n+1)/2 = (n+1)/2$$

With probability p that the key is present: $C_{avg}(n) = p(n+1)/2 + (1-p)\cdot n$. Preprocessing cost = 0, so for q searches $T_{lin}(n, q) = q \cdot \Theta(n)$.

Algorithm 2 – Binary search (sort first)

Each call makes one comparison and passes at most $\lfloor n/2 \rfloor$ records to the next call, so the comparison count satisfies $C(n) = C(\lfloor n/2 \rfloor) + 1$ (solved in Task 7). After k calls at most $n/2^k$ records remain; the search must stop when the range becomes empty, i.e. $n/2^k < 1$:

$$C_{worst}(n) = \lfloor \log_2 n \rfloor + 1$$

For n = 1,000,000 this is $\lfloor 19.93 \rfloor + 1 = 20$, which matches the maximum I measured (20). **Preprocessing:** comparison-based sorting needs $\Theta(n \log n)$ comparisons (Timsort, the built-in sort, is $O(n \log n)$ worst case). **Total for q searches:**

$$T_{bin}(n, q) = n \log_2 n + q(\lfloor \log_2 n \rfloor + 1)$$

Sorting pays for itself once $n \log n + q \log n < q \cdot n/2$ (average linear cost), that is $q >$ about $2 \log_2 n$. For $n = 10^6$ that is roughly 39 searches.

Algorithm 3 – Hash table with chaining

Building inserts n keys, each an O(1) append: $T_{build}(n) = n \cdot c = \Theta(n)$. Let m be the number of buckets and $\alpha = n/m$ the load factor (I use $\alpha = 0.75$). Assuming keys spread uniformly over buckets, a bucket holds α keys on average, so:

$$C_{unsuccessful} = \alpha = 0.75\ C_{successful} = 1 + \alpha/2 = 1.375$$

The second formula holds because a successful search finds its key after the 1 comparison for itself plus, on average, half of the other α keys that were inserted before it in the same chain. Neither depends on n. Total for q searches:

$$T_{hash}(n, q) = c \cdot n + q(1 + \alpha/2) = \Theta(n + q)$$

Combined cost (preprocessing + searching)

Algorithm	Preprocessing	One search (avg)	q searches in total
Linear	0	$\approx n/2$	$\approx q \cdot n/2$
Binary	$n \log_2 n$	$\approx \log_2 n$	$n \log_2 n + q \log_2 n$
Hash	n	≈ 1.375	$n + 1.375\ q$

Task 4 – Best, Average and Worst Case

	Best case	Average case	Worst case
Linear	Key is the first record: 1 comparison (the loop exits immediately).	Key present and equally likely anywhere: (n+1)/2 comparisons.	Key is last, or absent: all n records are examined, because nothing earlier tells the loop to stop.
Binary	Key is exactly the middle element of the sorted array: 1 comparison.	Successful search ≈ $\log_2 n - 1$ (most records sit in the deep levels of the implicit search tree, which has n/2 nodes at its last level); unsuccessful ≈ $\log_2 n$.	Key is absent or sits on the deepest level: $\lfloor \log_2 n \rfloor + 1$, since the range is halved each time and has to shrink to empty.
Hash	Key is the first item in an occupied bucket (or its bucket is empty for a missing key): 1 (0) comparisons.	$1 + \alpha/2 \approx 1.375$ successful; $\alpha = 0.75$ unsuccessful.	Every key hashes to the same bucket, so the chain has n items and the search degenerates to linear: n comparisons. With random registration numbers and a reasonable m this essentially does not occur; the longest chain I measured was short (see Task 9).

Measured at n = 1,000,000: linear best = 1, average = 584,752 (theory (n+1)/2 = 500,000), worst = 1,000,000; binary best = 1, average = 18.9 (theory ≈ 18.9), worst = 20; hash best = 1, average = 1.35 (theory 1.375), worst = 6. The counts follow the conditions explained above.

Task 5 – Asymptotic Analysis

Algorithm	Search time	Preprocessing	Meaning
Linear	O(n) worst, Θ(n) average, Ω(1) best	none	Cost is proportional to database size.
Binary	O(log n) worst, Θ(log n) average, Ω(1) best	Θ(n log n) once	Cost grows only with the logarithm of the size.
Hash	Θ(1) average, O(n) worst, Ω(1) best	Θ(n) once	Cost is independent of n on average; worst case needs a bad hash function.

Going from n = 1,000 to n = 1,000,000 (1000 times more records), the theory says linear search becomes ~1000 times slower, binary search only $\log_2(10^6)/\log_2(10^3) = 20/10 = 2$ times slower, and hash lookup stays about the same. My measured average comparison counts changed by factors of 1,162, 2.10 and 0.99 respectively. For the university, this means linear search is acceptable for a department-sized list but not for a million records, whereas the other two stay practical as the database keeps growing. The price of binary search is the Θ(n log n) sort and of hashing the Θ(n) build plus extra memory.

Task 6 – Formulate the Recurrence Relation

The recursive algorithm is binary search. I count key comparisons C(n), where n = hi − lo + 1 is the number of records still in the range.

Component	In my algorithm
Input size of the original problem	n = hi − lo + 1 (size of the current range)
Size of the recursive subproblem	at most ⌊n/2⌋: after comparing with A[mid], the call continues on A[lo..mid−1] or A[mid+1..hi], and each has at most ⌊n/2⌋ records
Number of recursive calls	1 (only one of the two halves is searched, never both)
Work outside the recursive call	computing mid and one key comparison = 1 basic operation, i.e. constant work

Component	In my algorithm
Base case	n = 0 (lo > hi): return −1 without any comparison. Using n = 1 as the base case gives C(1) = 1.

Putting these together (one call on a range of size ⌊n/2⌋, plus constant work):

$$C(n) = C(\lfloor n/2 \rfloor) + 1 \text{ for } n > 1,\ C(1) = 1$$

Task 7 – Solve the Recurrence Relation

Methods chosen: backward substitution (iteration) as the main derivation, and the Master Theorem as a check. Backward substitution is suitable because the recurrence has a single recursive term, so the pattern is easy to see once n is a power of two ($n = 2^k$); the Master Theorem applies because the recurrence has the standard form $T(n) = aT(n/b) + f(n)$.

Backward substitution ($n = 2^k$)

$$C(2^k) = C(2^{k-1}) + 1$$
$$= [C(2^{k-2}) + 1] + 1 = C(2^{k-2}) + 2$$
$$= C(2^{k-3}) + 3 = \ldots = C(2^{k-i}) + i$$

The pattern stops when $k - i = 0$, i.e. $i = k$. Then $C(2^k) = C(1) + k = 1 + k$. Since $k = \log_2 n$:

$$C(n) = \log_2 n + 1$$

For other n the floor function gives $C(n) = \lfloor \log_2 n \rfloor + 1$. *Check by induction:* if $C(\lfloor n/2 \rfloor) = \lfloor \log_2 \lfloor n/2 \rfloor \rfloor + 1 = \lfloor \log_2 n \rfloor - 1 + 1$, then $C(n) = \lfloor \log_2 n \rfloor - 1 + 1 + 1 = \lfloor \log_2 n \rfloor + 1$, which matches the formula.

Master Theorem check

Here $a = 1$, $b = 2$ and $f(n) = 1 = \Theta(n^0)$. Compare $f(n)$ with $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$. They have the same growth rate, which is Case 2 of the Master Theorem, so $T(n) = \Theta(n^{\log_b a} \cdot \log n) = \mathbf{\Theta(\log n)}$.

Interpretation. Each doubling of the database adds only one comparison. One million records need at most 20 comparisons and one billion would need only 30, so the recursive algorithm scales very well; the recursion depth is also only about log n, so the call stack never grows large (≤ 20 frames here).

Task 8 – Implement and Test the Algorithms

Implementation (all in `src/`): `algo1_linear.py` (iterative), `algo2_binary.py` (recursive, with a separate sort step), `algo3_hash.py` (own chained hash table, not Python's dict), `dataset_generator.py` and `benchmark.py`.

Test procedure (reproducible). (1) The dataset for each n ∈ {1,000; 10,000; 100,000; 1,000,000} is generated with random seed 1009, so all three algorithms receive the same records. (2) Records are unsorted, unique 9-digit registration numbers; my own register number 192511009 is always present and 192511010 is guaranteed absent. (3) Test keys: my reg no (found), reg no + 1 (not found), the first and last records, 20 random present keys and 20 random absent keys. (4) For every key all three algorithms must give the same found / not-found answer as Python's reference membership test; the program prints PASS or FAIL. (5) Timings use `time.perf_counter` on the same machine and process; comparison counts are collected by counters in the code.

Test case	Expected	Linear	Binary	Hash
192511009 (my reg no)	found	found	found	found
192511010 (reg no + 1)	not found	not found	not found	not found
first record / last record	found	found	found	found
20 random present, 20 random absent	found / not found	all correct	all correct	all correct

```
================================================================================
Reg No: 192511009 | Name: Avishek N | Seed: 1009 | Date/Time: 11-10-2026 18:45:49
Benchmark for n = 1,000
================================================================================

--- n = 1,000 ---
Correctness (all 3 algorithms agree with reference): PASS
Own reg no 192511009 -> linear idx 41, binary idx 243, hash found True
Reg no + 1 (192511010) -> linear -1, binary -1, hash found False
Algorithm   prep(ms)   found(us)  notfound(us)   best       avg     worst
Linear         0.000      20.187        41.488      1     503.2      1000
Binary         0.132       1.688         1.906      1       9.0        10
Hash           0.442       0.211         0.163      1       1.4         6
```

Figure 8.1: Program output (correctness test PASS) for n = 1,000

```
================================================================================
Reg No: 192511009 | Name: Avishek N | Seed: 1009 | Date/Time: 11-10-2026 18:45:49
Benchmark for n = 10,000
================================================================================

--- n = 10,000 ---
Correctness (all 3 algorithms agree with reference): PASS
Own reg no 192511009 -> linear idx 693, binary idx 2477, hash found True
Reg no + 1 (192511010) -> linear -1, binary -1, hash found False
Algorithm   prep(ms)   found(us)  notfound(us)   best       avg     worst
Linear         0.000     215.474       494.781      1    4777.2     10000
Binary         1.878       2.316         2.545      1      12.4        14
Hash           5.181       0.258         0.251      1       1.4         6
```

Figure 8.2: Program output (correctness test PASS) for n = 10,000

```
================================================================================
Reg No: 192511009 | Name: Avishek N | Seed: 1009 | Date/Time: 11-10-2026 18:45:49
Benchmark for n = 100,000
================================================================================

--- n = 100,000 ---
Correctness (all 3 algorithms agree with reference): PASS
Own reg no 192511009 -> linear idx 3612, binary idx 25003, hash found True
Reg no + 1 (192511010) -> linear -1, binary -1, hash found False
Algorithm   prep(ms)   found(us)  notfound(us)   best       avg     worst
Linear         0.000    8122.607     16384.443      1   54044.5    100000
Binary        27.132       4.054         4.113      1      15.6        17
Hash          63.993       0.663         0.526      1       1.4         6
```

Figure 8.3: Program output (correctness test PASS) for n = 100,000

```
================================================================================
Reg No: 192511009 | Name: Avishek N | Seed: 1009 | Date/Time: 11-10-2026 18:45:57
Benchmark for n = 1,000,000
================================================================================

--- n = 1,000,000 ---
Correctness (all 3 algorithms agree with reference): PASS
Own reg no 192511009 -> linear idx 11907, binary idx 251416, hash found True
Reg no + 1 (192511010) -> linear -1, binary -1, hash found False
Algorithm   prep(ms)   found(us)  notfound(us)   best       avg     worst
Linear         0.000  128445.049    220264.755      1  584752.3   1000000
Binary       530.711       5.944         6.434      1      18.9        20
Hash        2123.483       1.076         0.647      1       1.3         6
```

Figure 8.4: Program output (correctness test PASS) for n = 1,000,000

Task 9 – Record Experimental Performance

Times are the average over 1,000 random searches (200 for linear search up to n = 100,000 and 20 at n = 1,000,000, because each linear search on a million records is slow). Comparison counts are for successful searches. Raw data: `results/results.csv`.

Table 9.1: Average time per successful search (µs)

n	Linear search	Binary search	Hash table
1,000	20.19	1.69	0.21
10,000	215.47	2.32	0.26
100,000	8,122.61	4.05	0.66
1,000,000	128,445.05	5.94	1.08

Table 9.2: Average time per unsuccessful search (µs)

n	Linear search	Binary search	Hash table
1,000	41.49	1.91	0.16
10,000	494.78	2.54	0.25
100,000	16,384.44	4.11	0.53
1,000,000	220,264.76	6.43	0.65

Table 9.3: Average comparisons per successful search

n	Linear search	Binary search	Hash table
1,000	503.2	9.0	1.4
10,000	4,777.2	12.4	1.4
100,000	54,044.5	15.6	1.4
1,000,000	584,752.3	18.9	1.3

Table 9.4: Preprocessing time (ms)

n	Linear search	Binary search	Hash table
1,000	0.00	0.13	0.44
10,000	0.00	1.88	5.18
100,000	0.00	27.13	63.99
1,000,000	0.00	530.71	2,123.48

Table 9.5: Best / average / worst comparison counts

n	Linear best	Linear avg	Linear worst	Binary best	Binary avg	Binary worst	Hash best	Hash avg	Hash worst
1,000	1	503.2	1,000	1	9.0	10	1	1.4	6
10,000	1	4,777.2	10,000	1	12.4	14	1	1.4	6
100,000	1	54,044.5	100,000	1	15.6	17	1	1.4	6
1,000,000	1	584,752.3	1,000,000	1	18.9	20	1	1.3	6

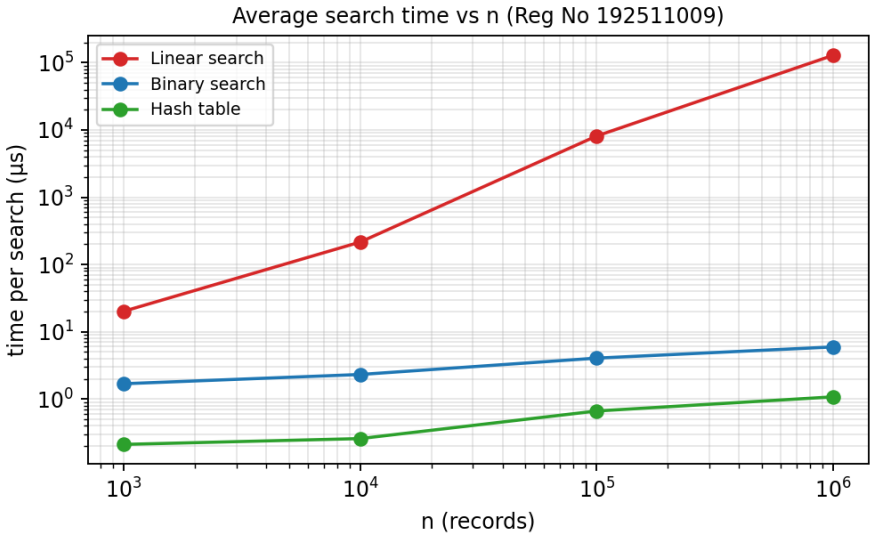


Figure 9.1: Average search time against n (log–log axes), generated from results.csv

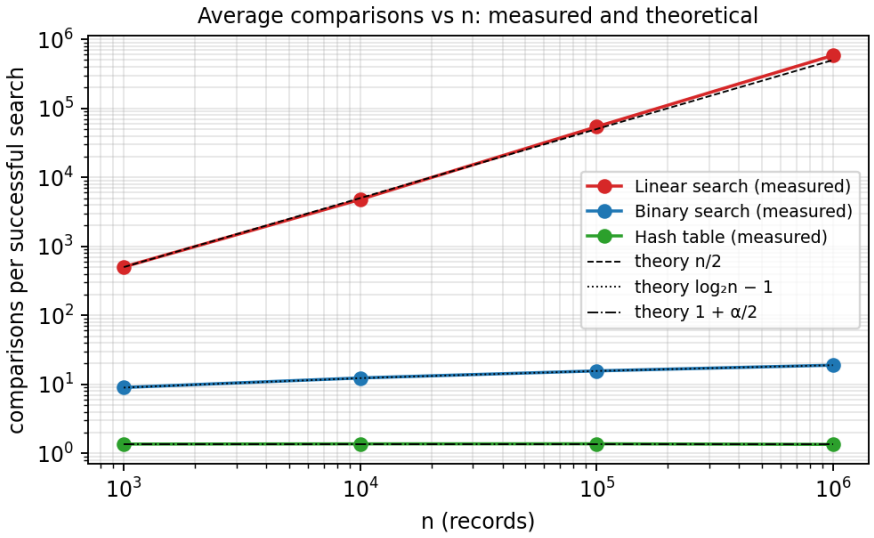


Figure 9.2: Average number of comparisons against n, with theoretical curves

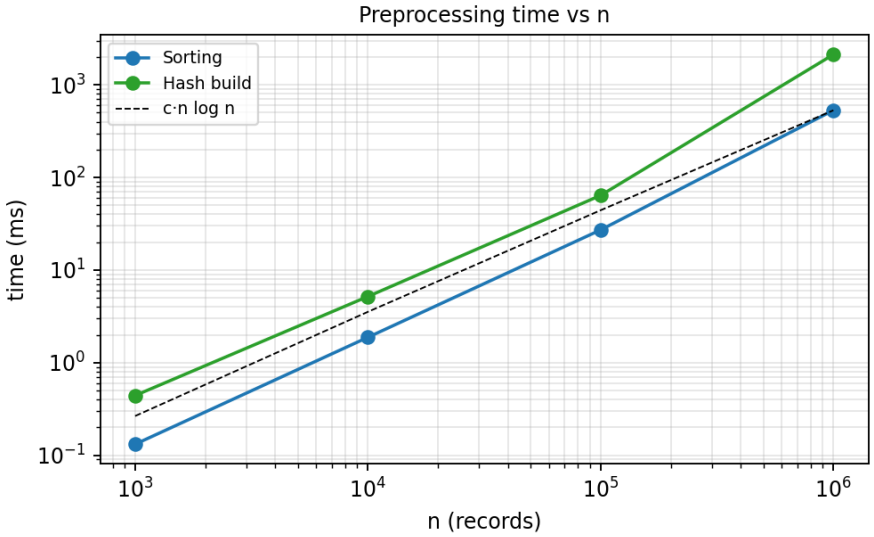


Figure 9.3: Preprocessing time (sorting and hash-table build) against n

```
================================================================================
Reg No: 192511009 | Name: Avishek N | Seed: 1009 | Date/Time: 11-10-2026 18:45:49
Benchmark for n = 1,000
================================================================================

--- n = 1,000 ---
Correctness (all 3 algorithms agree with reference): PASS
Own reg no 192511009 -> linear idx 41, binary idx 243, hash found True
Reg no + 1 (192511010) -> linear -1, binary -1, hash found False
Algorithm   prep(ms)   found(us)  notfound(us)   best       avg     worst
Linear         0.000      20.187        41.488      1     503.2      1000
Binary         0.132       1.688         1.906      1       9.0        10
Hash           0.442       0.211         0.163      1       1.4         6
```

Figure 9.4: Benchmark terminal output for n = 1,000

```
================================================================================
Reg No: 192511009 | Name: Avishek N | Seed: 1009 | Date/Time: 11-10-2026 18:45:49
Benchmark for n = 10,000
================================================================================

--- n = 10,000 ---
Correctness (all 3 algorithms agree with reference): PASS
Own reg no 192511009 -> linear idx 693, binary idx 2477, hash found True
Reg no + 1 (192511010) -> linear -1, binary -1, hash found False
Algorithm   prep(ms)   found(us)  notfound(us)   best       avg     worst
Linear         0.000     215.474       494.781      1    4777.2     10000
Binary         1.878       2.316         2.545      1      12.4        14
Hash           5.181       0.258         0.251      1       1.4         6
```

Figure 9.5: Benchmark terminal output for n = 10,000

```
================================================================================
Reg No: 192511009 | Name: Avishek N | Seed: 1009 | Date/Time: 11-10-2026 18:45:49
Benchmark for n = 100,000
================================================================================

--- n = 100,000 ---
Correctness (all 3 algorithms agree with reference): PASS
Own reg no 192511009 -> linear idx 3612, binary idx 25003, hash found True
Reg no + 1 (192511010) -> linear -1, binary -1, hash found False
Algorithm   prep(ms)   found(us)  notfound(us)   best       avg     worst
Linear         0.000    8122.607     16384.443      1   54044.5    100000
Binary        27.132       4.054         4.113      1      15.6        17
Hash          63.993       0.663         0.526      1       1.4         6
```

Figure 9.6: Benchmark terminal output for n = 100,000

```
================================================================================
Reg No: 192511009 | Name: Avishek N | Seed: 1009 | Date/Time: 11-10-2026 18:45:57
Benchmark for n = 1,000,000
================================================================================

--- n = 1,000,000 ---
Correctness (all 3 algorithms agree with reference): PASS
Own reg no 192511009 -> linear idx 11907, binary idx 251416, hash found True
Reg no + 1 (192511010) -> linear -1, binary -1, hash found False
Algorithm   prep(ms)   found(us)  notfound(us)   best       avg     worst
Linear         0.000  128445.049    220264.755      1  584752.3   1000000
Binary       530.711       5.944         6.434      1      18.9        20
Hash        2123.483       1.076         0.647      1       1.3         6
```

Figure 9.7: Benchmark terminal output for n = 1,000,000

Task 10 – Compare Theory with Experiment

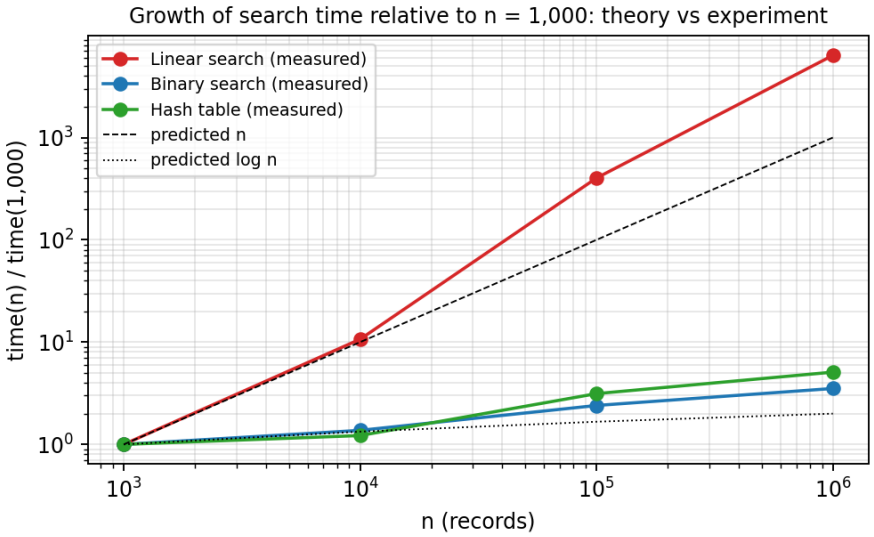


Figure 10.1: Measured growth of search time relative to n = 1,000, with predicted growth curves

Agreement. The comparison counts agree closely with the formulas (Figure 9.2): linear ≈ n/2 (584,752 vs 500,000), binary ≈ $\log_2 n - 1$ (18.9 vs 18.9), hash ≈ 1 + α/2 (1.35 vs 1.375). The slope of the log–log time curve, which is the observed growth exponent, is 1.30 for linear search (theory 1), 0.19 for binary search (theory ≈ 0, logarithmic) and 0.25 for the hash table (theory 0).

Which scales better. The hash table scales best, then binary search, and linear search is far behind: at n = 1,000,000 a linear search takes 128.4 ms, binary search 5.9 µs and the hash lookup 1.08 µs, i.e. linear search is about 21,610 times slower than binary search.

Differences from theory and possible reasons. (1) Each 10× increase in n should make linear search about 10× slower. In my runs the factors were 10.7×, 37.7×, 15.8×. Factors well above 10 are probably caused by the list of Python integer objects no longer fitting in the CPU cache, so every comparison gets slower; this is a hardware effect, not a change in the algorithm. (2) Binary search and hash times may creep upward slowly although their comparison counts are almost flat; cache misses are again a likely cause, since each probe jumps to a distant memory location. (3) The recursive binary search pays function-call overhead in Python, a constant factor that counting comparisons does not capture. (4) Timer resolution and other running processes add noise at the microsecond scale. None of these change the shape of the curves, so the experiment supports the theoretical analysis.

Task 11 – Recommend and Defend the Final Design

Recommendation for the original problem: binary search on a sorted array.

- **Why not linear search:** its cost is proportional to n. At one million records one search takes about 128 ms and examines about 584,752 records on average.
- **Why binary search:** Θ(log n) comparisons (≈ 19 on average, at most 20) and about 5.9 µs per search at $n = 10^6$. It needs no extra memory beyond the array, and the sorted order also supports range queries (e.g. all registration numbers of one batch) and ordered reports, which the hash table cannot do.
- **Preprocessing cost:** sorting 10^6 records took 531 ms, which is repaid after only about 39 searches (Task 3).
- **Why not the hash table as the default:** it is the fastest (1.08 µs) but its build was 4.0 times slower than sorting (2,123 ms), it uses extra memory for buckets and chains, and its worst case is O(n) with a poor hash function.
- **Trade-offs and limits:** inserting into a sorted array costs O(n) per record, so new records must be added in batches and the array re-sorted (or merged). The hash table's advantage grows with

the number of searches, which is exactly what Task 12 examines.

- **Growth:** doubling the database adds one comparison, so the design stays practical far beyond 10^6 records.

Task 12 – Day-4 Design Challenge

A. Impact analysis

Originally the number of searches was unspecified, so cost per search mattered little compared with the effort of setting up the structure. Now the load is fixed: 50,000 searches per hour on 1,000,000 records, which is 13.9 searches per second, or one request every 72 ms. The number of searches is now important because the preprocessing cost can be shared by 1,200,000 searches per day (50,000 × 24) while the search cost is paid every time. Because new records arrive only once a day, the data are effectively static between updates: the sorted array or the hash table can be built once per day and reused, and the O(n) cost of inserting into a sorted array — normally the weak point of binary search — never appears during the day. The assumption that is affected is that preprocessing cost has to be balanced against a small number of searches; it no longer has to.

B. Evaluation of the original design

The original recommendation, sorted array + binary search, takes 5.65 µs per search on this machine. All 50,000 searches take 0.28 s, plus the daily sort of 0.58 s, far below the hour available. The preprocessing cost is therefore acceptable: 0.58 s once a day is negligible. For comparison, linear search needs about 132 ms per search, which is longer than the 72 ms gap between requests; 50,000 searches would need about 6,586 s ≈ 1.8 h of computing for one hour of requests (extrapolated from 100 real searches). The queue would grow without limit, so linear search is ruled out. A hash table is the only alternative that could give better search performance: 0.97 µs per search.

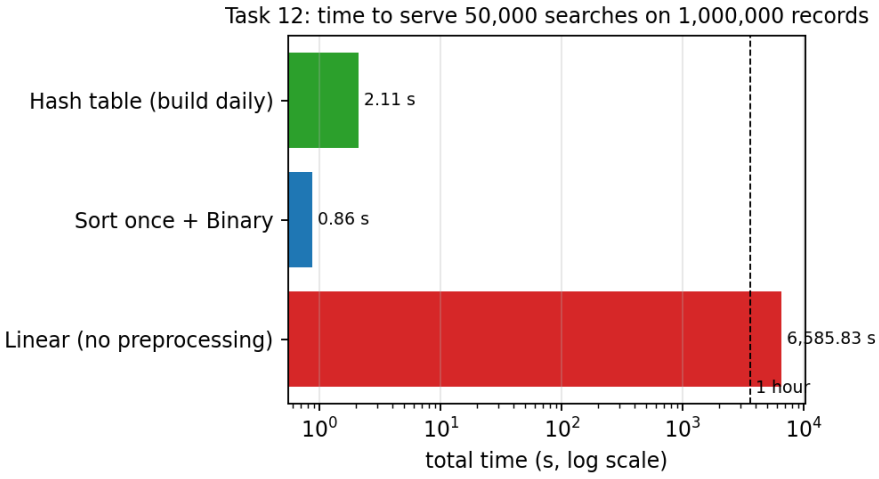


Figure 12.1: Total time (daily preprocessing + 50,000 searches) for each strategy at n = 1,000,000

Strategy	Preprocessing (s)	50,000 searches (s)	Per search (µs)	How measured
Linear (no preprocessing)	0.000	6,585.830	131,716.59	extrapolated from 100 real searches
Sort once + Binary	0.582	0.282	5.65	measured, all 50,000
Hash table (build daily)	2.058	0.049	0.97	measured, all 50,000

C. Adaptation or redesign: MODIFY

I decided to **modify**, not replace, the original design. The algorithm (binary search) stays the same, but the data handling changes: (1) the sort is turned into a nightly batch job that runs when the new records arrive; (2) the new day's records are sorted separately (k log k) and merged into the existing sorted array in O(n + k) instead of re-sorting everything in O(n log n); (3) all daytime searches use the read-only sorted array, so no locking or per-insert cost is needed. Effect on complexity: search stays Θ(log n); the amortised preprocessing is Θ(n + k log k) per day, i.e. about 0.485 µs per search spread over the day's 1,200,000 requests, which is negligible.

D. Evidence-based defence

- **Mathematics:** binary search costs $\lfloor \log_2 n \rfloor + 1 = 20$ comparisons at $n = 10^6$; the daily sort is $\Theta(n \log n) \approx 2 \times 10^7$ comparisons, spread over 1.2 million searches per day.
- **Experiment:** measured 0.28 s for all 50,000 searches (hash table: 0.05 s; linear search: about 6,586 s, extrapolated).
- **Search frequency:** the required rate is 13.9 per second, while binary search handles about 177,144 per second on one core, i.e. 12,744 times the demand.
- **Why not the hash table:** it is about 5.8 times faster per search, but that saving is irrelevant when binary search already uses well under 1 % of one core; the hash build (2.06 s vs 0.58 s for sorting) and its memory use are the price. If the request rate grew by a factor of several hundred, the hash table would become the better choice, and the design allows that switch because both structures are rebuilt in a nightly batch.
- **Trade-offs:** a modified sorted-array design keeps ordered/range queries and low memory use, at the cost of a nightly batch update (new students become searchable only after the update, which the problem statement allows).

Final decision: keep binary search, modify the preprocessing into a daily merge-and-sort batch; linear search is rejected, and a hash index is the planned upgrade if the load grows.

Evidence for Task 12 is in `results/task12.csv` and `results/log_task12.txt`.

```
================================================================================
Reg No: 192511009 | Name: Avishek N | Seed: 1009 | Date/Time: 11-10-2026 18:46:20
Task 12 design challenge
================================================================================

=== TASK 12: 50,000 searches on 1,000,000 records ===
Strategy                      prep(s)  50k searches(s)  per search(us)  note
Linear (no preprocessing)       0.000         6585.830       131716.59  extrapolated from 100 real searches
Sort once + Binary              0.582            0.282            5.65  measured, all 50,000
Hash table (build daily)        2.058            0.049            0.97  measured, all 50,000
```

Figure 12.2: Task 12 benchmark output (50,000 searches on 1,000,000 records)